

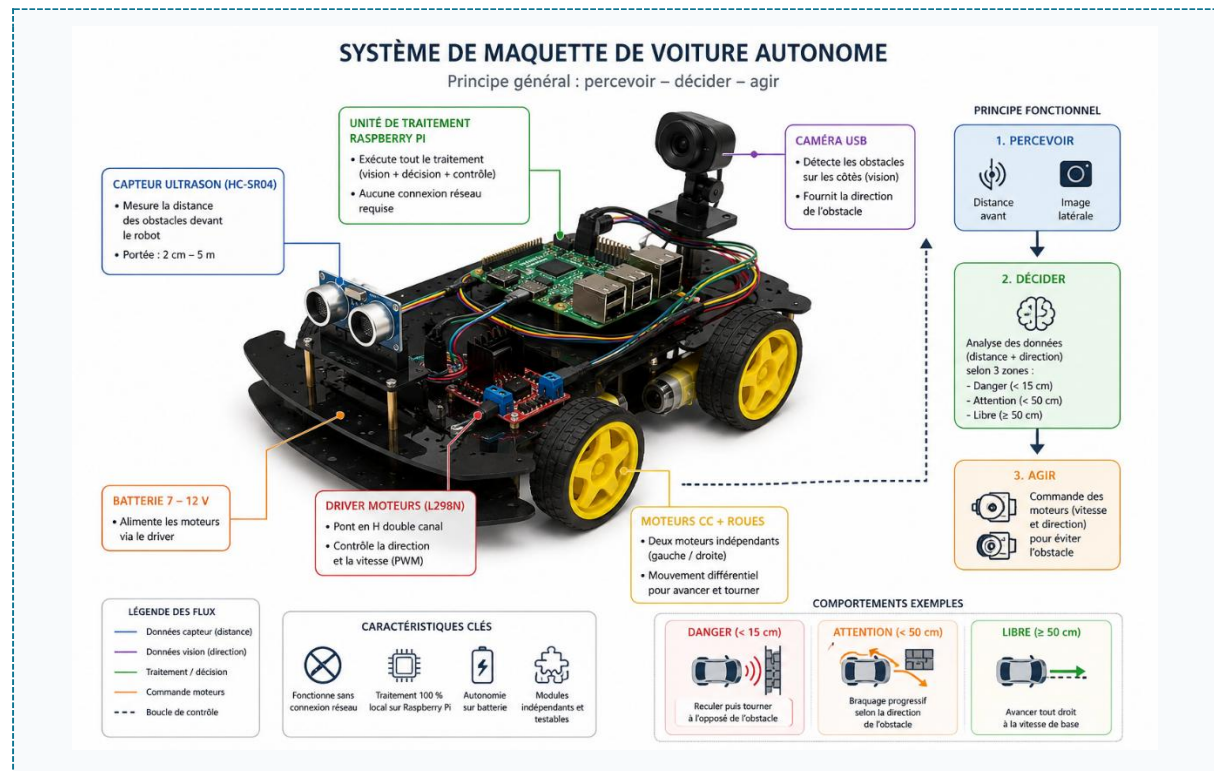
PROJET ROBOTIQUE · RASPBERRY PI

Pilotage Intelligent

Robot autonome d'évitement d'obstacles

GOBERT Yanis

MARVIN Leo



Plateforme : Raspberry Pi | Langage : Python | OpenCV | Matériel : HC-SR04

Table des matières

1. Présentation du système	3
Objectifs	3
2. Architecture logicielle	3
3. Matériel nécessaire	3
4. Câblage et montage	4
4.1 Brochage GPIO (numérotation BCM)	4
4.2 Pont diviseur sur la ligne ECHO	5
5. Le driver L298N : un pont en H	6
Direction, vitesse et puissance	6
6. Fonctionnement du code	6
6.1 Mesure de distance — ultrasonic.py	6
6.2 Détection visuelle — detector.py	7
6.3 Commande des moteurs — motors.py	7
6.4 Logique de décision — main.py	7
7. Mise en service	8
7.1 Tester chaque élément séparément	8
7.2 Lancement	8
8. Dépannage	9
9. Conclusion et perspectives	9
Pistes d'amélioration	9

1. Présentation du système

Ce projet met en œuvre un **robot mobile autonome** capable de détecter et de contourner les obstacles en temps réel. Il combine deux sources de perception complémentaires — un capteur ultrason qui mesure la **distance** à l'obstacle et une caméra qui en estime la **position latérale** — pour piloter deux moteurs à courant continu via un pont en H L298N.

La logique est progressive : le robot avance en ligne droite tant que la voie est libre, adapte sa trajectoire dès qu'un obstacle approche, et exécute une manœuvre d'urgence en cas de danger immédiat. Tout le traitement s'exécute en local sur le Raspberry Pi, sans GPU ni connexion réseau.

Objectifs

- **Percevoir** : mesurer la distance aux obstacles et repérer leur position latérale.
- **Décider** : choisir le comportement adapté selon le niveau de risque.
- **Agir** : piloter les deux moteurs pour éviter l'obstacle sans s'arrêter.

2. Architecture logicielle

Le code suit un découpage clair en quatre modules indépendants, organisés selon la chaîne **perception** → **décision** → **action**. Ce découplage permet de tester chaque élément isolément avant l'intégration.

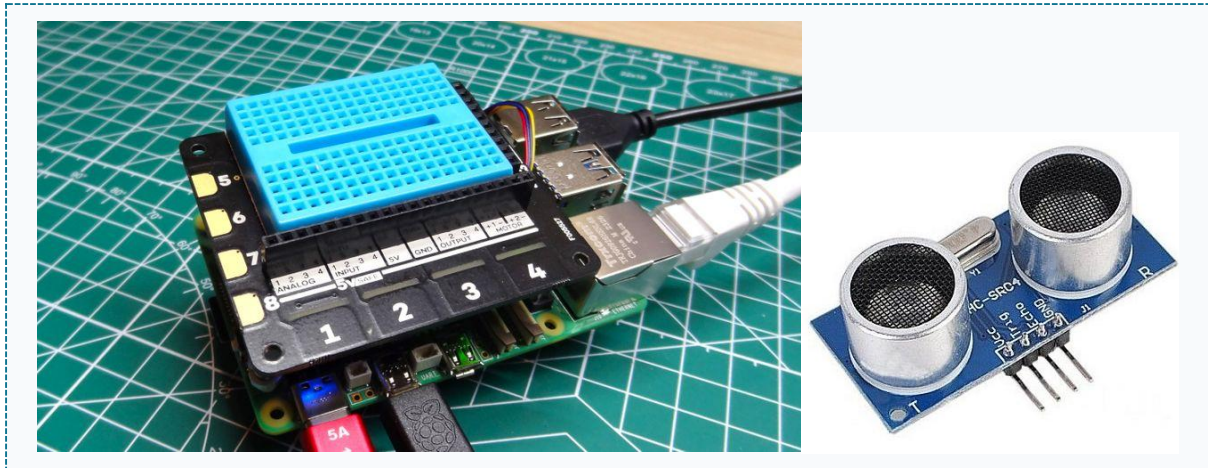
Module	Rôle	Points clés
ultrasonic.py	Perception — distance	HC-SR04, temps de vol, portée ~5 m
detector.py	Perception — direction	Caméra USB + OpenCV, biais dans [-1 ; +1]
main.py	Décision	Boucle temps réel, logique à trois zones
motors.py	Action	Driver L298N, vitesses PWM différentielles

3. Matériel nécessaire

La liste ci-dessous récapitule les composants nécessaires à la réalisation du robot.

Composant	Rôle	Qté
Raspberry Pi 4 / 5	Cerveau : traitement et commande	1
Capteur HC-SR04	Mesure de distance par ultrasons	1
Caméra USB	Détection latérale des obstacles	1
Driver L298N	Pont en H : commande des moteurs	1
Moteurs CC + roues	Propulsion gauche / droite	2
Batterie 7-12 V	Alimentation des moteurs	1

Composant	Rôle	Qté
Résistances 1 k Ω + 2 k Ω	Pont diviseur sur la ligne ECHO	2
Châssis, fils, breadboard	Structure et câblage	—



4. Câblage et montage

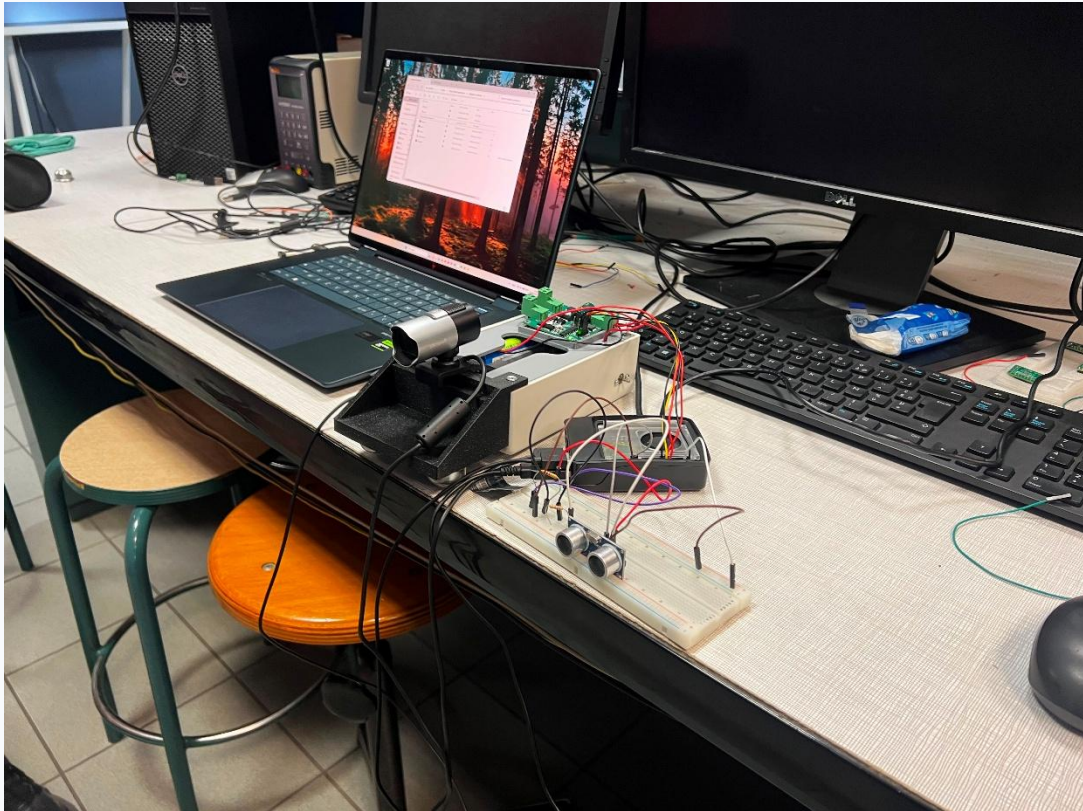
4.1 Brochage GPIO (numérotation BCM)

Le tableau ci-dessous détaille toutes les connexions entre le Raspberry Pi et les périphériques.

Broche Pi	Composant	Fonction
GPIO 5	HC-SR04 TRIG	Déclenche l'impulsion (branchement direct)
GPIO 6	HC-SR04 ECHO	Retour de l'écho — via pont diviseur
GPIO 17 / 27	L298N IN1 / IN2	Sens du moteur gauche
GPIO 22 / 23	L298N IN3 / IN4	Sens du moteur droit
GPIO 18	L298N ENA	Vitesse PWM — moteur gauche
GPIO 24	L298N ENB	Vitesse PWM — moteur droit
5 V / GND	HC-SR04 / L298N	Alimentation logique et masse commune

Trois règles évitent la plupart des problèmes à cette étape :

- **Masse commune** : le GND du Pi, celui du L298N et celui de la batterie doivent tous être reliés ensemble.
- **Alimentations séparées** : les moteurs sont alimentés par la batterie via le L298N, jamais par le Pi.
- **Adaptation de tension** : seule la ligne ECHO doit être abaissée de 5 V à 3,3 V (voir section 4.2).

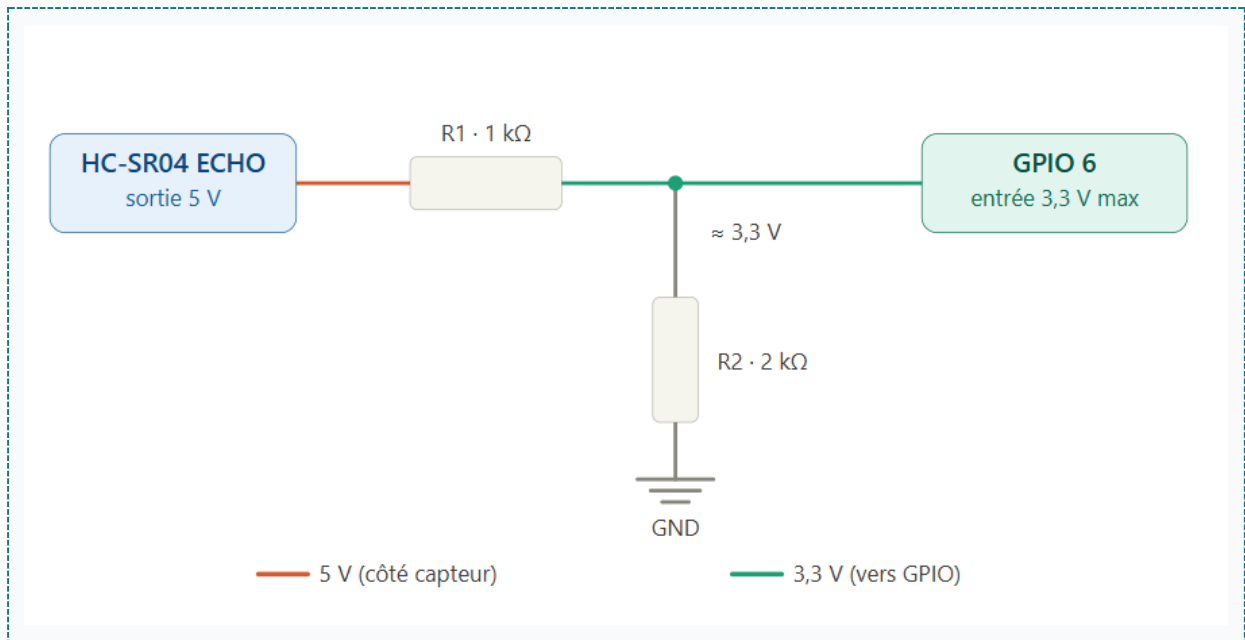


4.2 Pont diviseur sur la ligne ECHO

La broche ECHO du HC-SR04 délivre un signal à **5 V**, alors que les entrées GPIO du Raspberry Pi tolèrent **3,3 V au maximum**. Un pont diviseur composé de deux résistances abaisse cette tension :

1. Placer **R1 (1 kΩ)** en série depuis la broche ECHO du capteur.
2. L'autre extrémité de R1 forme le **point milieu**, d'où part le fil vers **GPIO 6**.
3. Brancher **R2 (2 kΩ)** entre ce point milieu et la **masse**.

La tension au point milieu vaut alors $V = 5 \times R2 / (R1 + R2) = 5 \times 2 / 3 \approx 3,3 \text{ V}$, pile sous la limite du Pi. À noter que la broche TRIG, elle, se branche directement : le Pi envoie 3,3 V, accepté tel quel par le capteur.



5. Le driver L298N : un pont en H

Le L298N est un **double pont en H** : deux circuits identiques et indépendants, un par moteur. Chaque pont est formé de quatre interrupteurs (des transistors) entourant le moteur. Fermer une paire diagonale fait circuler le courant dans un sens ; fermer l'autre diagonale l'inverse.

Direction, vitesse et puissance

- **Direction** : les entrées IN fixent le sens. **IN1=HIGH**, **IN2=LOW** donne la marche avant ; la combinaison inverse donne la marche arrière.
- **Vitesse** : un signal **PWM** sur les broches ENA / ENB règle la vitesse. Un rapport cyclique de 60 % correspond à environ 60 % de la vitesse maximale.
- **Puissance** : les moteurs tirent leur courant de la batterie, le L298N ne fait que le commuter selon les ordres logiques du Pi.

Deux remarques pratiques : le L298N **chauffe** et fait chuter la tension d'environ 1,5 à 2 V (prévoir une batterie un peu supérieure à la tension nominale des moteurs), et il intègre déjà les **diodes de protection** contre les pics de tension des bobinages.

6. Fonctionnement du code

6.1 Mesure de distance — ultrasonic.py

Le capteur fonctionne par **temps de vol** : il émet une impulsion et mesure le délai de retour de l'écho. La distance se déduit de cette durée et de la vitesse du son (343 m/s), divisée par deux pour l'aller-retour.

```
def distance_cm(self):
    GPIO.output(TRIG, GPIO.HIGH)
    time.sleep(0.00001) # impulsion 10 us
    GPIO.output(TRIG, GPIO.LOW)
    while GPIO.input(ECHO) == 0: # attente echo
        start = time.time()
    while GPIO.input(ECHO) == 1: # duree echo
        end = time.time()
    return (end - start) * 34300 / 2 # cm
```

6.2 Détection visuelle — detector.py

La caméra indique **où** se trouve l'obstacle. Une soustraction de fond adaptative isole les objets, le plus grand contour est retenu, et sa position horizontale est convertie en un **bias** dans [-1 ; +1] : négatif à gauche, positif à droite.

```
Mask = self._bg.apply(band) # soustraction de fond
contours, _ = cv2.findContours(mask,
    cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cx = int(M['m10'] / M['m00']) # centre du contour
bias = (cx - FRAME_W / 2) / (FRAME_W / 2)
```

6.3 Commande des moteurs — motors.py

Le signe de la vitesse détermine le sens de rotation, sa valeur absolue le rapport cyclique PWM (donc la vitesse). La classe `MotorDriver` initialise elle-même les broches et le PWM à sa création.

```
def _set_motor (self, in1, in2, pwm, speed):
    speed = max (-100, min(100, speed))
    if speed >= 0: # marche avant
        GPIO.output(in1, GPIO.HIGH)
        GPIO.output(in2, GPIO.LOW)
    else: # marche arriere
        GPIO.output(in1, GPIO.LOW)
        GPIO.output(in2, GPIO.HIGH)
    pwm.ChangeDutyCycle(abs(speed))
```

6.4 Logique de décision — main.py

La boucle principale choisit son comportement selon la distance mesurée, répartie en trois zones :

Zone	Condition	Action
Danger	distance < 15 cm	Stop, marche arrière 0,5 s, puis demi-tour à l'opposé
Attention	distance < 50 cm	Évitement progressif : biais caméra, braquage proportionnel
Libre	distance ≥ 50 cm	Marche avant en ligne droite à la vitesse de base (60 %)

Le cœur du braquage est la fonction `compute_wheel_speeds` : l'écart de vitesse entre les roues combine la **direction** (biais) et l'**urgence** (proximité). Plus l'obstacle est proche, plus la correction est forte.

```

proximity = (WARNING_CM - distance) /
             (WARNING_CM - DANGER_CM) # 0 -> 1
delta = bias * TURN_FACTOR * proximity
left  = BASE_SPEED + delta             # roue gauche
right = BASE_SPEED - delta             # roue droite

```

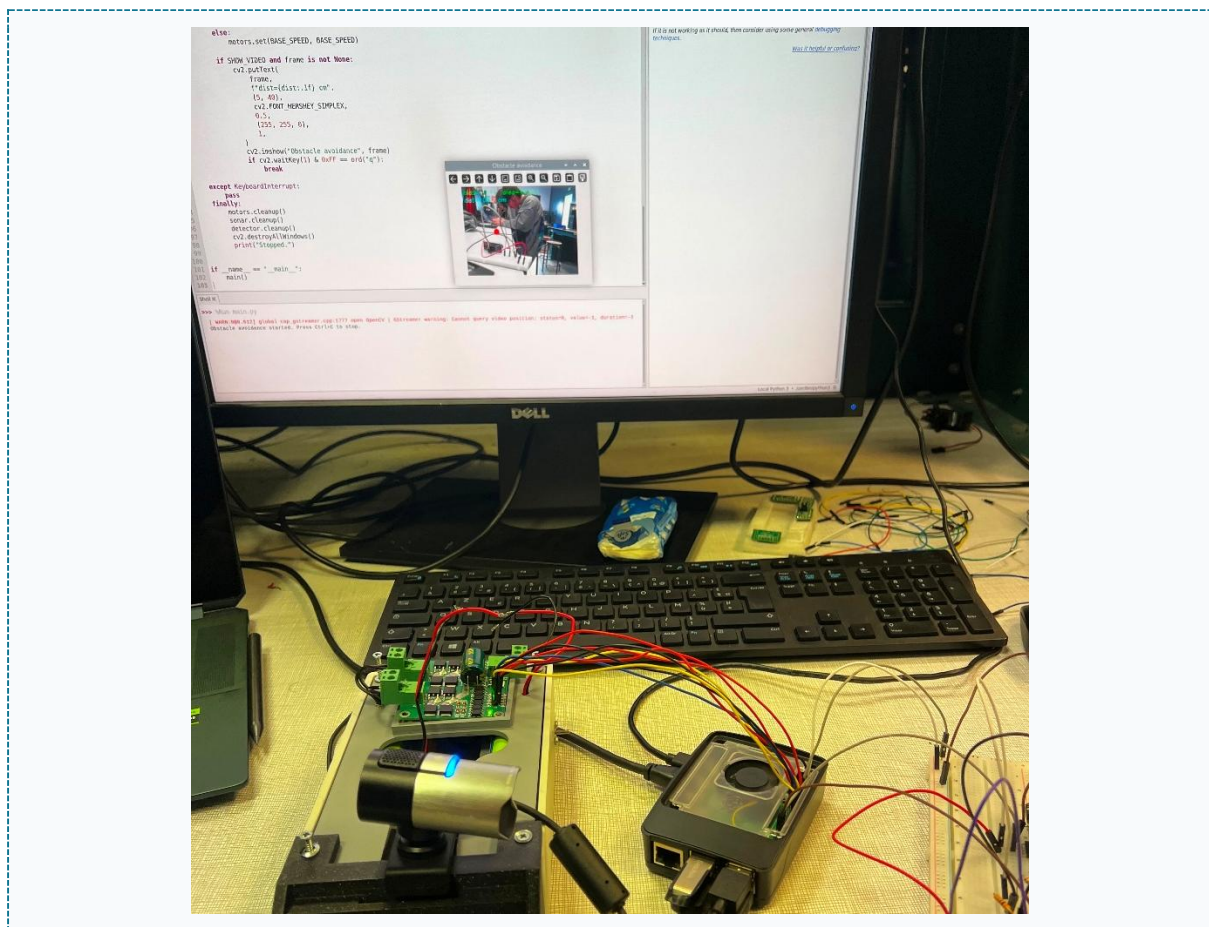
7. Mise en service

7.1 Tester chaque élément séparément

Avant de lancer la boucle complète, on vérifie capteur et moteurs isolément. **Surélevez le robot** (roues dans le vide) pour le test des moteurs.

- `test_ultrasonic.py` : affiche la distance en continu — la valeur doit suivre la main devant le capteur.
- `test_motors.py` : fait tourner chaque moteur en avant puis en arrière — vérifie le sens et la correspondance gauche / droite.

Si un moteur tourne à l'envers, il suffit d'**inverser ses deux fils de sortie** sur le bornier du L298N — aucun code à modifier.



7.2 Lancement

Une fois les tests concluants, lancer la boucle complète depuis le dossier du projet :


```
cd robot
python3 main.py

# arrêt propre : Ctrl + C
```

Sur un Pi piloté sans écran (en SSH), passer la ligne `SHOW_VIDEO = False` dans `main.py` : sinon l'affichage vidéo provoque une erreur faute d'écran. Le reste de la logique fonctionne sans problème dans ce mode.

8. Dépannage

Les problèmes qui ont été rencontrés ainsi que leur résolution :

Symptôme	Cause et solution
Cannot determine SOC peripheral base address	RPi.GPIO incompatible (Pi 5 / Bookworm). Installer le remplaçant <code>rpi-lgpio</code> (aucun changement de code).
Le capteur renvoie toujours l'infini	Vérifier le pont diviseur, l'alimentation 5 V du capteur et la masse commune.
Un moteur tourne à l'envers	Inverser ses deux fils de sortie sur le bornier du L298N.
Moteurs erratiques ou sans réponse	Masse commune manquante entre le Pi, le L298N et la batterie.
Moteur qui couine sans tourner	Batterie trop faible, ou chute de tension du L298N : augmenter la tension d'alimentation.

9. Conclusion et perspectives

Le système démontre qu'un évitement d'obstacles robuste peut reposer sur des briques simples et peu coûteuses, à condition de bien **combiner** leurs informations et de **doser** la réaction selon le risque. L'architecture modulaire laisse une marge confortable pour les améliorations futures.

Pistes d'amélioration

- **Régulation PID** sur le braquage pour lisser la trajectoire et éviter les oscillations.
- **Détection par apprentissage** (YOLO embarqué) pour fiabiliser la reconnaissance des obstacles.
- **Multi-capteurs** : plusieurs HC-SR04 pour élargir le champ de perception.
- **Filtrage des mesures** (moyennage glissant) afin de réduire le bruit du capteur de distance.

SYSTÈME DE MAQUETTE DE VOITURE AUTONOME

Principe général : percevoir – décider – agir

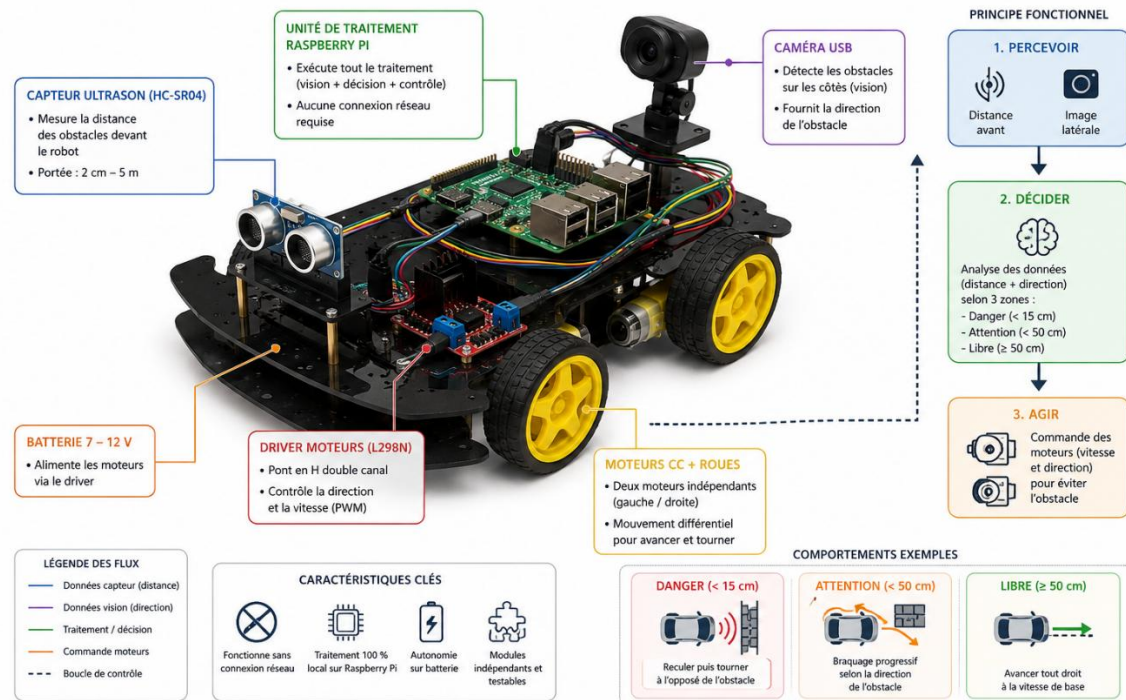


Schéma résumant le principe de la maquette développée